

7/10

Testing / Pruebas

Proceso [PUD]

- Requerimientos
- Análisis
- Diseño
- Implementación
- Pruebas
- Despliegue

Reporte de Defectos

Uso del Producto

Testing

• Testing Agile hace un nuevo enfoque,

se lo define como una Actividad Distribuida

Encuentra defectos cuya presencia se presume que hay en el código

Etapas

Artefacto

Pruebas

Es bastante extensa



1. Planificación

→ Plan de Pruebas

2. Diseño

→ Casos de Pruebas

3. Ejecución

← Reporte de Defectos

(List de cosas que no funcionan)

4. Cierre

En la US definimos los escenarios por después ver el caso de pruebas realizado

Se lo presentamos a los desarrolladores

Cuando el producto cumple con la calidad.

No tiene sentido de esperar la versión del producto, para empezar a planificar.

Entonces la planificación y el diseño (con el 50% del esfuerzo)

no puedo realizar antes y de esta manera acorto el tiempo.

Los puedo adelantar hasta después de definir los requerimientos.

Entonces

Un plan responde a la pregunta que, como, cuando, quien? (Las mismas preguntas por cualquier ser el plan).

! Los Artefactos tienen que estar sin importar el enfoque (Agile o Tradicional, o etc).

CONCEPTOS

* Testing metodológico vs Ad-hoc

↓
El que se hace
en proceso

No metodológico
No sistematizado, no tiene proceso
Sin casos de prueba.

↓
Detecta errores
(pero son irreproducibles)

- No tiene casos de prueba que te vayan diciendo
que pasa según.

- No hay documentación, no tenemos los requerimientos

↓
Si no hay casos de prueba no tiene sentido probar.

Prueba

→ Validar: Es un proceso que tiene que ver con la recolección
Veras si hace lo que tiene que hacer.

→ Verificación: Lo que hace, lo hace bien?

→ Creamos casos de prueba a partir del código voy a poder
verificar pero no puedo validar (por que no se que tiene que hacer).

El error de requerimiento es cero.

Error vs Defecto

↳ Falta

Un error o defecto que provoca una falta software
Nosotros

un mal funcionamiento en el sistema

• Algo es un defecto cuando se trata de una etapa anterior a la actual (previa)

Testigo encuentra defectos.

• Error cuando no se trata de una burla no pasan a otra etapa

Costo
↑
↓
basado

Errores Requerimiento
= Arquitectura
= Implementación

! Defecto hay que es una actividad intensiva humano-intensiva

• El testing se lleva entre el 30% a 50% del ESFUERZO → y esto es el 80% del costo

• Implementación no deben llevar más del 35% del esfuerzo.

• No se hace testing suficiente, el testing exhaustivo no se puede pagar el esfuerzo mucho más y no estos porcentajes

Edad de Defecto = Tiempo que transcurre desde que se inserta hasta que se soluciona.

↳ cuanto más edad más caro

Para programming → la versión más económica, detecta el error a el costo.

que la programador es.

Categorías de los Defectos → tiene que ver con la Severidad de Defecto

- ↳ Bloqueante o Inutilizante: No se puede usar el sistema
- ↳ Grave
- ↳ Leve
- ↳ Cosmético
- ↳ Mejor

→ Consejo, no es un defecto. Creo que se podría hacer mejor

Defecto asigna severidad y estado.

Error se cometen → ortografía
Formato del campo
Interfaz

La severidad no tiene que ver
cuanto la cuesta el equipo
concepción.

Tiene que ver con el impacto
que produce en el sistema

La severidad le asigna el testing pero

deben acordarlo con el cliente (product owner) y desarrolladores.

• Prioridad: Urgencia del cliente (lo define el)

{ Alta
Media
Baja

Niveles de Testing.

Ayuda a la validación

Se prueba de lo
más chico a lo más
grande

→ Pruebas Unitarias (TDD resuelve este → se hace en implementación)

→ Pruebas Integración o de Interfaz

↳ Como funcionan los componentes entre los solo ya sabemos como ellos

→ Sistema

entrega

se termina en build

↳ Pruebas de Aceptación

Sistema e I



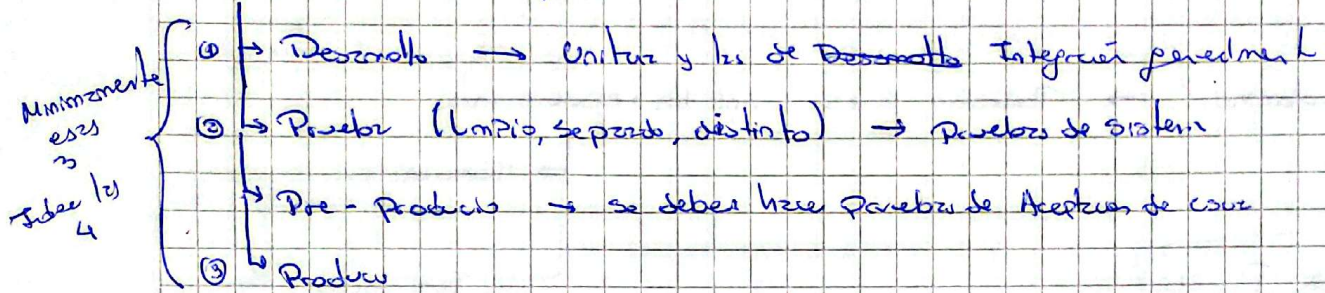
Si o si
en Pruebas

Pruebas de Aceptación → en el despliegue.

Un desarrollo o unidad de trabajo no puede probar su desarrollo.

↳ Pruebas unitarias si o si el que lo desarrolla

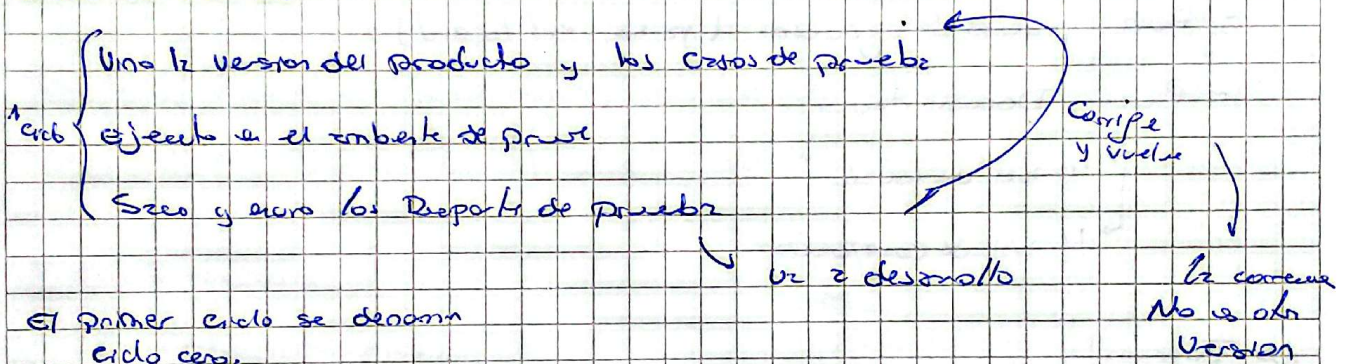
Ambientes de Prueba Software



Los de Desarrollo no tienen permiso de ingreso al ambiente de Prueba.

Si desarrollo software como tercero, no tenemos de producción.

Ciclo de Prueba



El primer ciclo se denomina ciclo cero.

Se repite hasta que este OK.

Cuántos ciclos tiene que haber?

↳ Idealmente 2. → por cada versión que entra a testing

Ciclo de Prueba

↳ Estrategia de prueba

↳ Ejecuto el ciclo o, reporte, → Desarrollo y devuelve y crea tests en punto de decisión

Sin Regresión → Tomando el reporte y veo si se corrigió

Con Regresión → Hago todos los casos de prueba, desde cero

Per puede haber nuevo problema y no se detecta
"error" "error" "outlets"

La automatización de pruebas permite realizar con la Regresión, y se hace por tiempo o costo.

Que probamos? → Probamos para cumplir los requerimientos

Tipos de Pruebas

→ En que se enfoca en probz.

Por Funcionalidad

- Pruebas Reg. operación Normal (los casos)
- Pruebas Negativas (los casos de fallo)
- Pruebas proceso de negocio. (ya son end to end)
- Pruebas Documentación
 - Manual de usuario
 - Manual de configuración

→ Funcionales:

→ No Funcionales: Son difíciles porque el ambiente de pruebas debe ser el mismo nivel de producción

↓
La mayoría se hace ~~pruebas~~ en el ambiente de producción.

No Funcionales

- Pruebas de Performance → Tiempo de respuesta. → Usuario e usuario concurrentes
 - Pruebas de Interfaz → Combinación que puede realizar el usuario en pantalla
 - Pruebas de Escala Completa → Todo lo que yo me comprometo en infraestructura
→ usuarios concurrentes
empres
 - Pruebas de estrés
 - = Seguridad
 - = Accesibilidad
- Probar con más de lo que me comprometo
Como levantarse a lo importante.
- Log. n
Disponibilidad
Alta en entorno ruidoso

También se pueden probar en Aprobación, y sistema. Pero no todos se pueden

Probar a nivel de integración y sistema

El que crea un defecto es testing, el estado lo crean los testers.

La etapa a veces no se dificulta porque desarrollador no puede no solucionarlo.

Caso de Prueba

↳ contiene descripción detallada que se va a hacer validación, verificación de una funcionalidad

Tiene:

- un escenario (que es un paso a paso)

- Datos de prueba → Precondiciones están incluidas en los datos

- Resultado esperado (ya definido en el user).

Cuando el sistema está funcionando, es bueno hacer otros de producción, reales y procesarlos (mejorarlo, ...) pero no viola la confidencialidad.

! Son Defectos

↳ Datos Específicos

~~Erro~~

Defectos conocidos, lo se pero no lo vamos a corregir

Cuando vamos a dejar de probar → se define en el plan de prueba el criterio

↳ Adición de defectos

↳ De la cantidad de tiempo que queda (personal)

↳ Pasa a métrica concreta (% de pruebas, líneas, casos y 0% de bugs).

↳ Cuando ~~sean~~ corren los casos de prueba

• Testing no es en condiciones de asegurar la calidad

• Testing no pone calidad, hace evidente la falta de calidad

• Testing hace control de calidad, no aseguramiento